

# generative adversarial network (GAN)

stats403\_deep\_learning  
spring\_2025  
lecture\_8

## 8.1 generative adversarial networks (GAN)

# What is “generative”?

- when we “generate” a data instance, we are sampling from the distribution of data
- let  $x$  denote data instances and  $y$  the labels
  - ❑ a discriminative model learns  $p(y|x)$
  - ❑ a generative model learns  $p(x, y)$  or  $p(x)$

# generative models

- Boltzmann machine:

$$P(\mathbf{x}) = \frac{\exp(-E(\mathbf{x}))}{Z} \qquad E(\mathbf{x}) = -\mathbf{x}^\top \mathbf{U} \mathbf{x} - \mathbf{b}^\top \mathbf{x}$$

- divide the random variable into visible and hidden units

$$E(\mathbf{v}, \mathbf{h}) = -\mathbf{v}^\top \mathbf{R} \mathbf{v} - \mathbf{v}^\top \mathbf{W} \mathbf{h} - \mathbf{h}^\top \mathbf{S} \mathbf{h} - \mathbf{b}^\top \mathbf{v} - \mathbf{c}^\top \mathbf{h}$$

# generative models

- restricted Boltzmann machine (RBM)

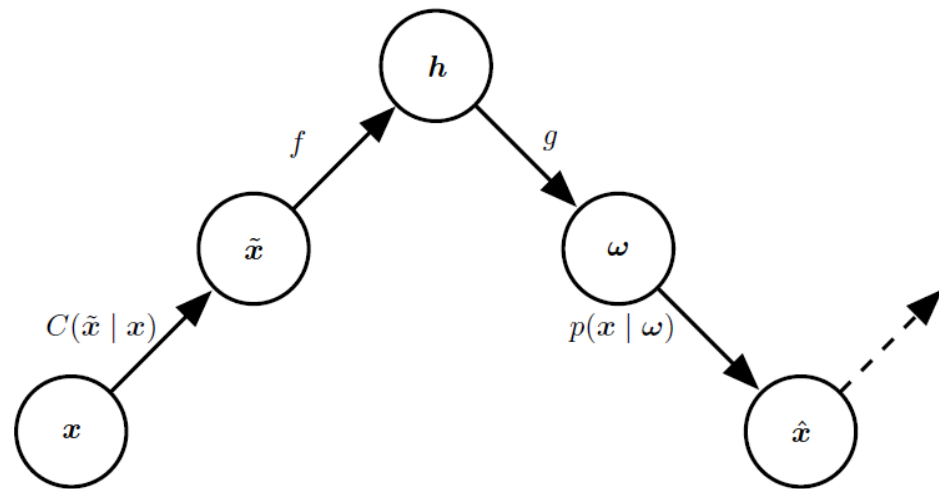
$$P(\mathbf{v} = \mathbf{v}, \mathbf{h} = \mathbf{h}) = \frac{1}{Z} \exp(-E(\mathbf{v}, \mathbf{h}))$$

$$E(\mathbf{v}, \mathbf{h}) = -\mathbf{b}^\top \mathbf{v} - \mathbf{c}^\top \mathbf{h} - \mathbf{v}^\top \mathbf{W} \mathbf{h}$$

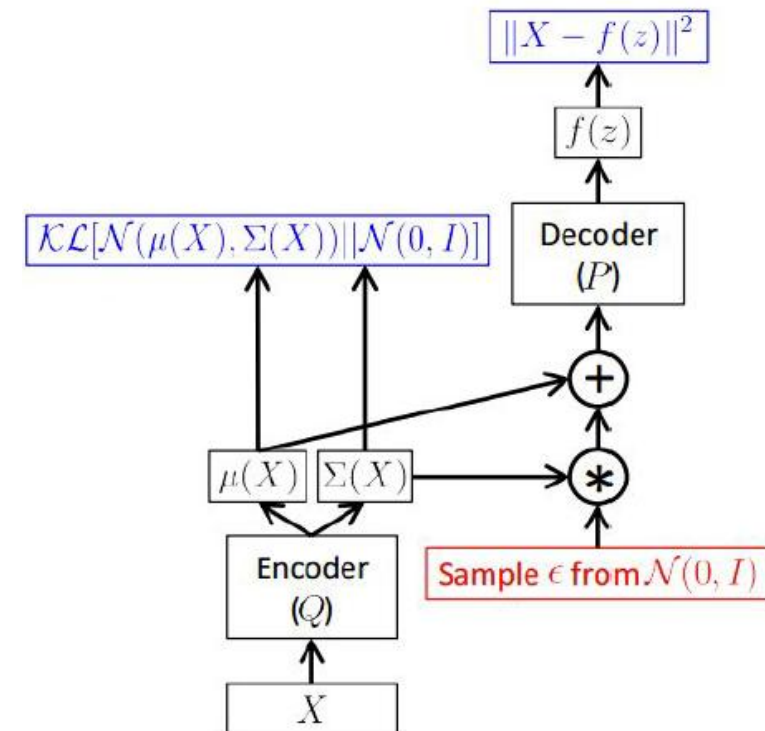
- <http://www.cs.toronto.edu/~hinton/absps/guideTR.pdf> for a practical guide for training RBMs.

# generative models

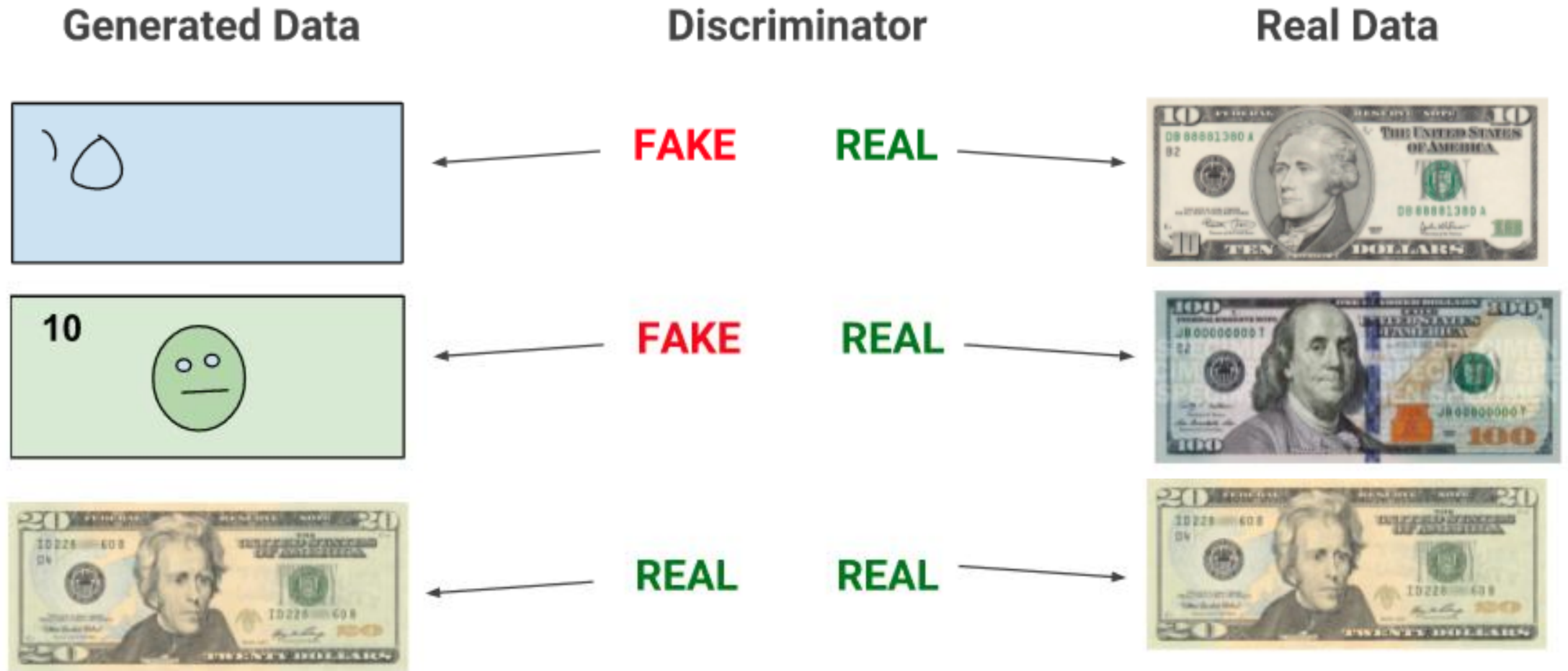
- Auto-Encoders



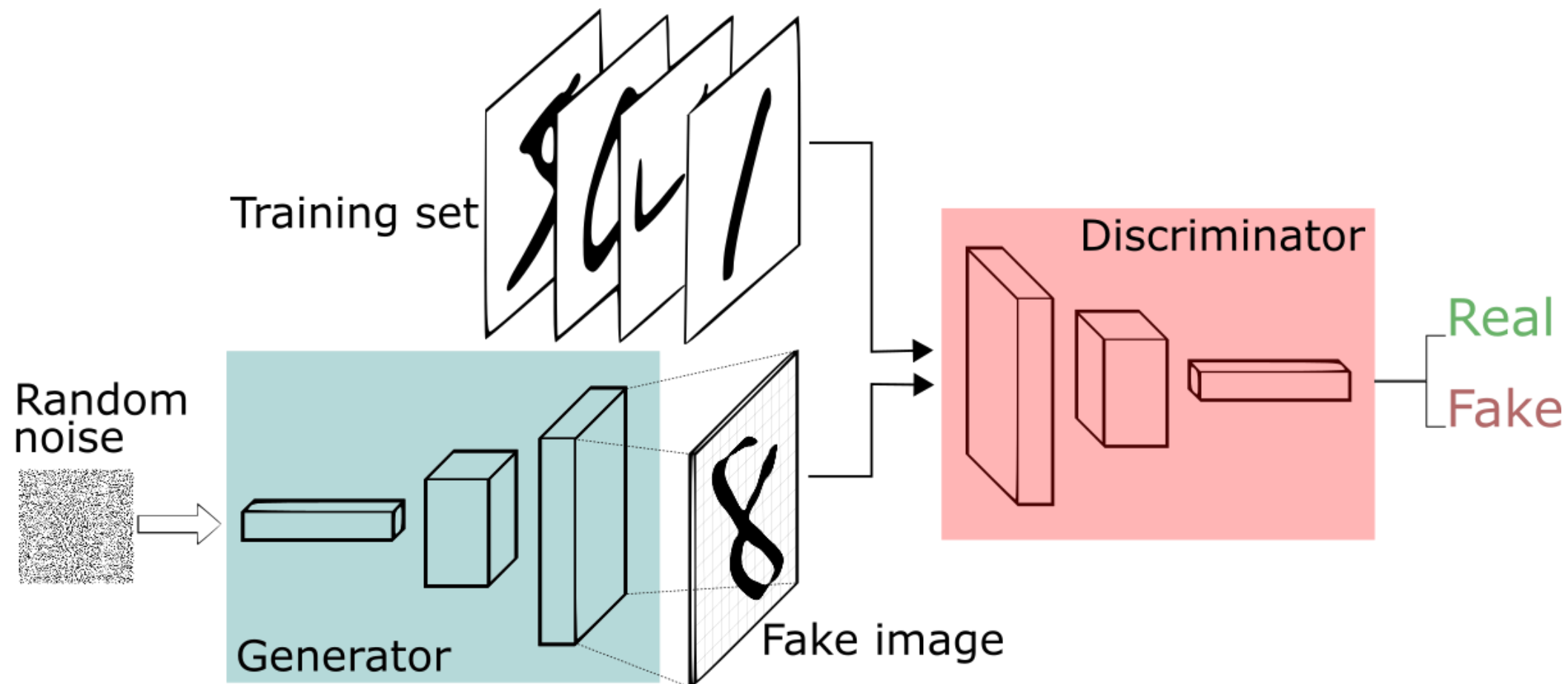
- Variational Auto-Encoders



# generative adversarial network (GAN)

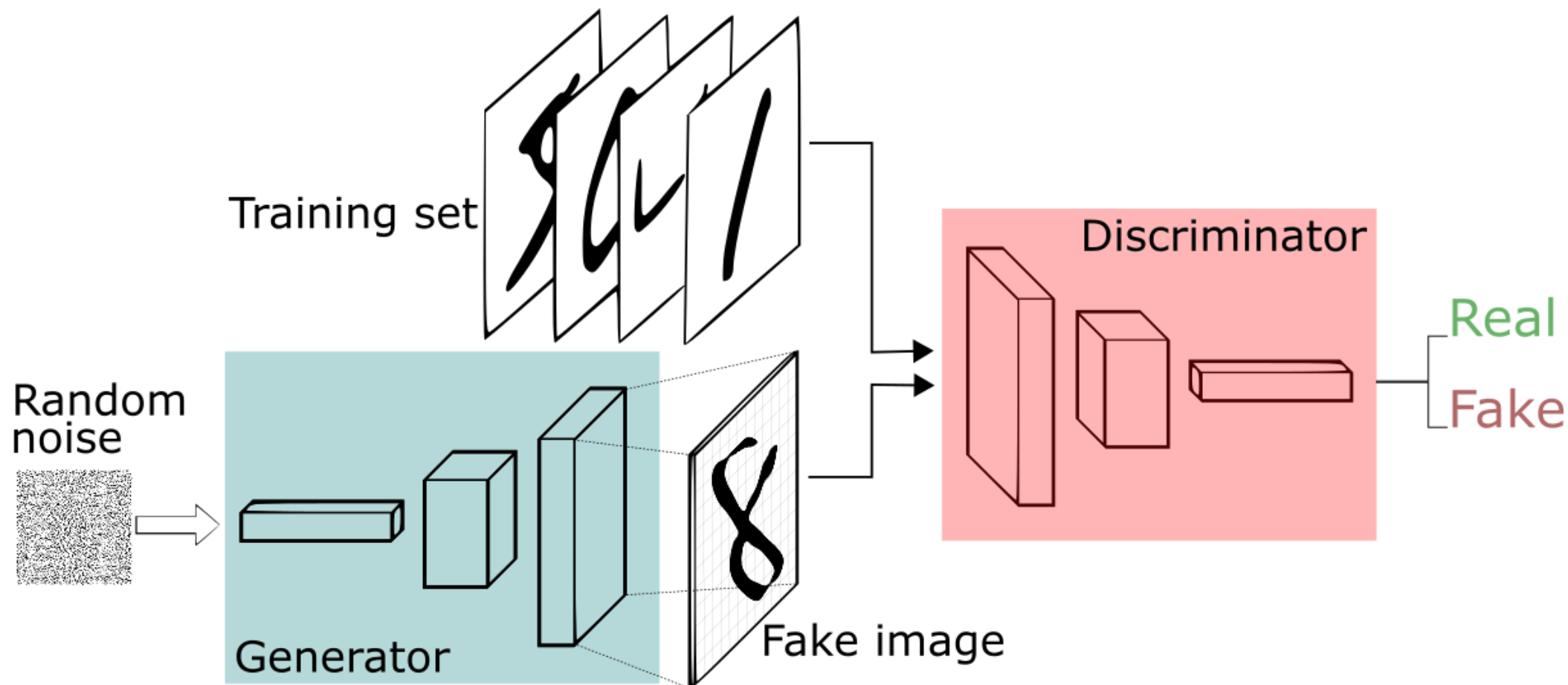


# GAN



# GAN

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))].$$

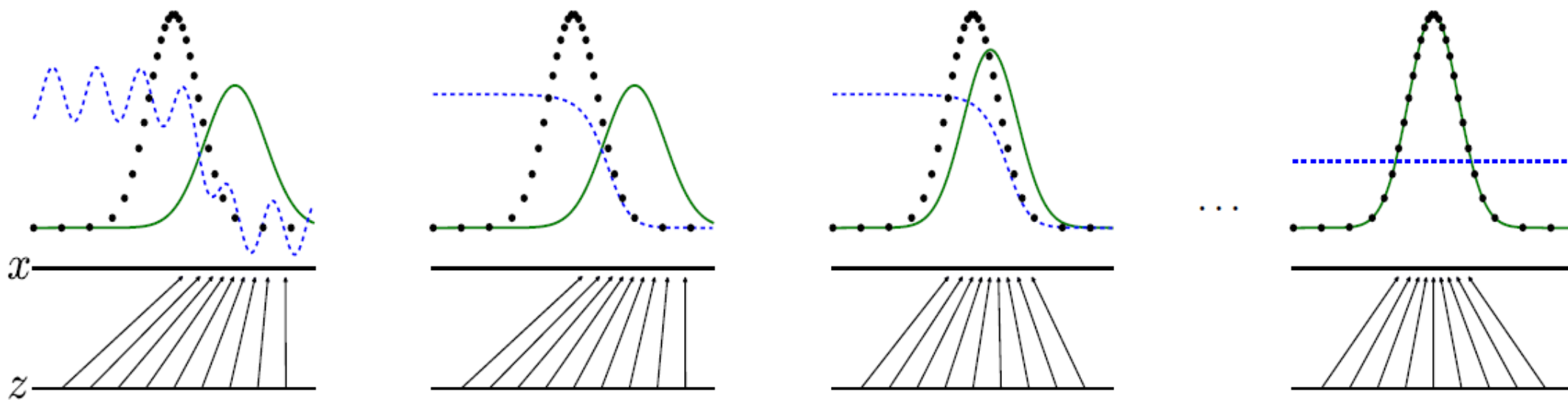


# development of GANs

- Goodfellow, I., ...& Bengio, Y. (2014). Generative adversarial nets. *NIPS*.
- Arjovsky, M., & Bottou, L. (2017). Towards principled methods for training generative adversarial networks. *ICLR*.
- Arjovsky, M., Chintala, S., & Bottou, L. (2017). Wasserstein generative adversarial networks. *ICML*.
- Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., & Courville, A. (2017). Improved training of Wasserstein GANs. *NIPS*.

# training GAN

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))].$$



# training GAN

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))] .$$

**for** number of training iterations **do**

**for**  $k$  steps **do**

- Sample minibatch of  $m$  noise samples  $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$  from noise prior  $p_g(\mathbf{z})$ .
- Sample minibatch of  $m$  examples  $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$  from data generating distribution  $p_{\text{data}}(\mathbf{x})$ .
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[ \log D(\mathbf{x}^{(i)}) + \log(1 - D(G(\mathbf{z}^{(i)}))) \right] .$$

**end for**

- Sample minibatch of  $m$  noise samples  $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$  from noise prior  $p_g(\mathbf{z})$ .
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log(1 - D(G(\mathbf{z}^{(i)}))) .$$

**end for**

# training GAN

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))].$$

- for a fixed generator  $G$ , the optimal discriminator  $D$  is (why?)

$$D_G^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_g(\mathbf{x})}$$

- if we can reach the optimal  $D$ , what is the objective function in  $G$ ?

## training GAN

$$= \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} \left[ \log \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_g(\mathbf{x})} \right] + \mathbb{E}_{\mathbf{x} \sim p_g} \left[ \log \frac{p_g(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_g(\mathbf{x})} \right]$$

- the global minimum of  $C(G)$  is achieved if and only if  $p_g = p_{\text{data}}$

# fGAN

- f-divergence 
$$D_f(P\|Q) = \int_{\mathcal{X}} q(x) f\left(\frac{p(x)}{q(x)}\right) dx$$

□f: a convex, lower-semicontinuous function with  $f(1) = 0$

| Name              | $D_f(P\ Q)$                                                                                 | Generator $f(u)$                       |
|-------------------|---------------------------------------------------------------------------------------------|----------------------------------------|
| Kullback-Leibler  | $\int p(x) \log \frac{p(x)}{q(x)} dx$                                                       | $u \log u$                             |
| Reverse KL        | $\int q(x) \log \frac{q(x)}{p(x)} dx$                                                       | $-\log u$                              |
| Pearson $\chi^2$  | $\int \frac{(q(x)-p(x))^2}{p(x)} dx$                                                        | $(u-1)^2$                              |
| Squared Hellinger | $\int \left(\sqrt{p(x)} - \sqrt{q(x)}\right)^2 dx$                                          | $(\sqrt{u}-1)^2$                       |
| Jensen-Shannon    | $\frac{1}{2} \int p(x) \log \frac{2p(x)}{p(x)+q(x)} + q(x) \log \frac{2q(x)}{p(x)+q(x)} dx$ | $-(u+1) \log \frac{1+u}{2} + u \log u$ |

# fGAN

- convex conjugate

$$f^*(t) = \sup_{u \in \text{dom}_f} \{ut - f(u)\}$$

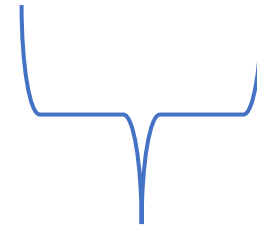
$$f(u) = \sup_{t \in \text{dom}_{f^*}} \{tu - f^*(t)\}$$

$$\begin{aligned} D_f(P \| Q) &= \int_{\mathcal{X}} q(x) \sup_{t \in \text{dom}_{f^*}} \left\{ t \frac{p(x)}{q(x)} - f^*(t) \right\} \mathrm{d}x \\ &\geq \sup_{T \in \mathcal{T}} \left( \int_{\mathcal{X}} p(x) T(x) \mathrm{d}x - \int_{\mathcal{X}} q(x) f^*(T(x)) \mathrm{d}x \right) \\ &= \sup_{T \in \mathcal{T}} (\mathbb{E}_{x \sim P} [T(x)] - \mathbb{E}_{x \sim Q} [f^*(T(x))]) , \end{aligned}$$

# fGAN

- an objective for generator  $Q$

$$F(\theta, \omega) = \mathbb{E}_{x \sim P} [T_\omega(x)] - \mathbb{E}_{x \sim Q_\theta} [f^*(T_\omega(x))]$$



two neural networks

# fGAN

more generally, a **divergence**  $D$  w.r.t. a space  $S$  of probability distributions with common support is a function

$$D(\cdot \| \cdot) : S \times S \rightarrow \mathbb{R}$$

such that

1.  $D(p \| q) \geq 0, \forall p, q \in S$
2.  $D(p \| q) = 0$ , iff  $p = q$

## difficulty with GAN

- If  $D$  is trained too well,  $D$  may not provide gradient for  $G$ .
- Specifically, if data and generated instances have disjointed supports, then there is a  $D^*$  that takes value 1 for data and value 0 for generated instances. This  $D^*$  is constant on each support and the gradient thus vanishes.
- Even if  $D$  is not exactly but close to  $D^*$ , there is a problem.

## difficulty with GAN

- Suppose  $\|D - D^*\| < \epsilon$  and  $\mathbb{E}_{z \sim p(z)} [\|J_\theta g_\theta(z)\|_2^2] \leq M^2$
- For the generator,

$$\begin{aligned} \|\nabla_\theta \mathbb{E}_{z \sim p(z)} [\log(1 - D(g_\theta(z)))]\|_2^2 &\leq \mathbb{E}_{z \sim p(z)} \left[ \frac{\|\nabla_\theta D(g_\theta(z))\|_2^2}{|1 - D(g_\theta(z))|^2} \right] \\ &\leq \mathbb{E}_{z \sim p(z)} \left[ \frac{\|\nabla_x D(g_\theta(z))\|_2^2 \|J_\theta g_\theta(z)\|_2^2}{|1 - D(g_\theta(z))|^2} \right] \\ &< \mathbb{E}_{z \sim p(z)} \left[ \frac{(\|\nabla_x D^*(g_\theta(z))\|_2 + \epsilon)^2 \|J_\theta g_\theta(z)\|_2^2}{(|1 - D^*(g_\theta(z))| - \epsilon)^2} \right] \\ &= \mathbb{E}_{z \sim p(z)} \left[ \frac{\epsilon^2 \|J_\theta g_\theta(z)\|_2^2}{(1 - \epsilon)^2} \right] \\ &\leq M^2 \frac{\epsilon^2}{(1 - \epsilon)^2} \end{aligned}$$

## difficulty with GAN

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))].$$

- rather than training  $G$  to minimize  $\log(1 - D(G(\mathbf{z})))$  we can train  $G$  to maximize  $\log D(G(\mathbf{z}))$

## difficulty with GAN

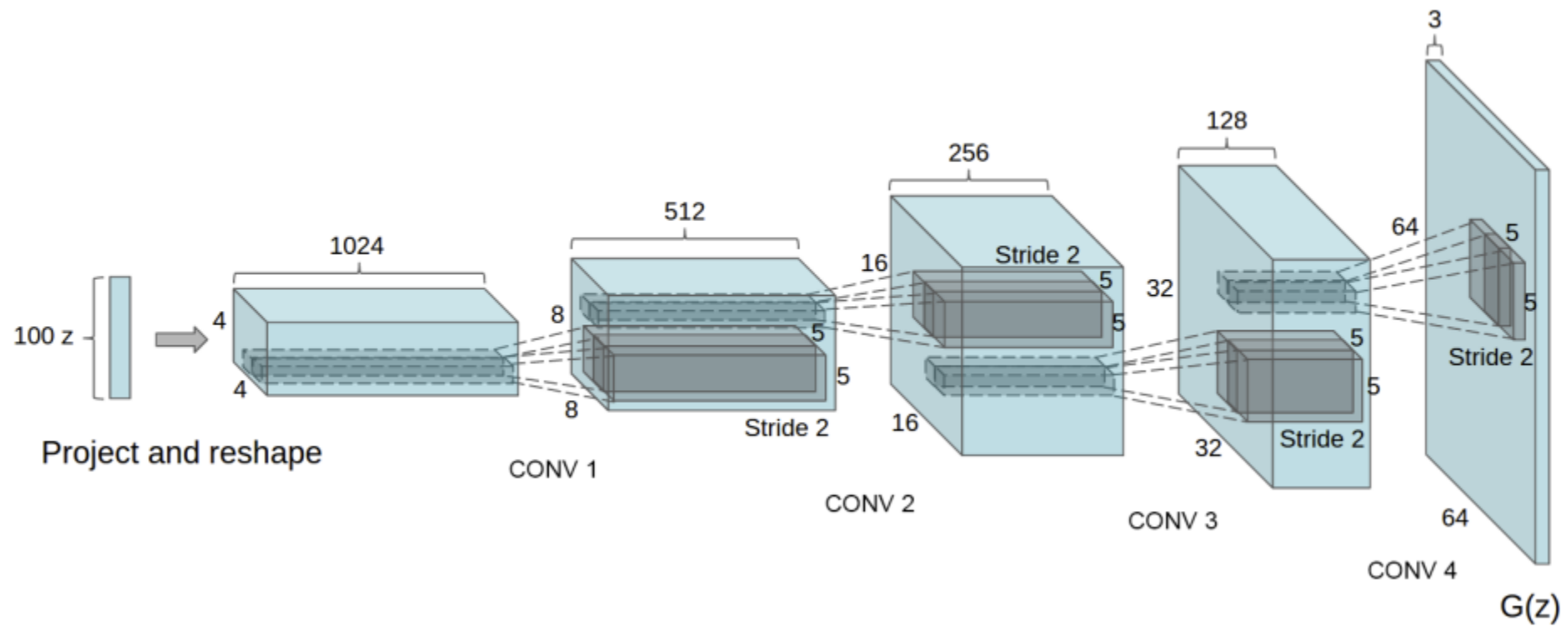
- if data and generated instances are disjointed, then

$$JSD(\mathbb{P}_r \parallel \mathbb{P}_g) = \log 2$$

$$KL(\mathbb{P}_r \parallel \mathbb{P}_g) = +\infty$$

$$KL(\mathbb{P}_g \parallel \mathbb{P}_r) = +\infty$$

# DCGAN



# DCGAN

- [https://pytorch.org/tutorials/beginner/dcgan\\_faces\\_tutorial.html](https://pytorch.org/tutorials/beginner/dcgan_faces_tutorial.html)
- Let's play:
  - <https://colab.research.google.com/drive/1eXMCbgUnDaGrw-dhXMDQgreMd5qSW45M?usp=sharing>

# measure the quality of generation

- inception score (IS)
  - measures the quality of the generated images by passing through a trained classifier
- Fréchet inception distance (FID)
  - measures the W2 distance between data and generated instances as Gaussian distributions

$$\text{FID} = |\mu - \mu_w|^2 + \text{tr}(\Sigma + \Sigma_w - 2(\Sigma\Sigma_w)^{1/2})$$

## 8.2 Wasserstein GAN (WGAN)

## review: difficulty with GAN

- if  $D$  is trained too well,  $D$  may not provide gradient for  $G$
- specifically, if data and generated instances have disjointed supports, then there is a  $D^*$  that takes value 1 for data and value 0 for generated instances
- this  $D^*$  is constant on each support and the gradient thus vanishes
- even if  $D$  is not exactly but close to  $D^*$ , there is a problem

## review: problem with divergence

- if data and generated instances are disjoint, then

$$JSD(\mathbb{P}_r \parallel \mathbb{P}_g) = \log 2$$

$$KL(\mathbb{P}_r \parallel \mathbb{P}_g) = +\infty$$

$$KL(\mathbb{P}_g \parallel \mathbb{P}_r) = +\infty$$

## a different notion of “distance” between distributions

- we defined the “distance” using “divergence”
- can we define a distance in the following way?
  - the distance between  $p$  and  $q$  is the minimum work required to warp  $p$  so that it becomes  $q$

$$\min_T \int c(x, T(x)) p(x) dx : T\#p = q$$

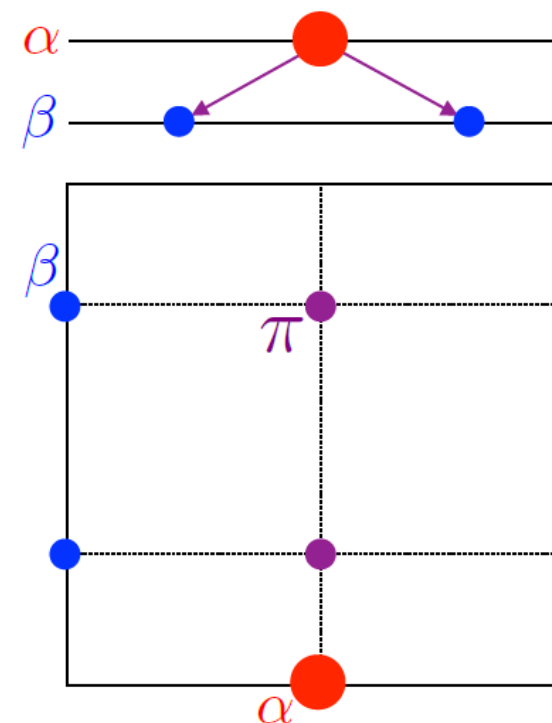
# Monge's problem

- the optimization problem

$$\min_T \int c(x, T(x)) p(x) dx : T\#p = q$$

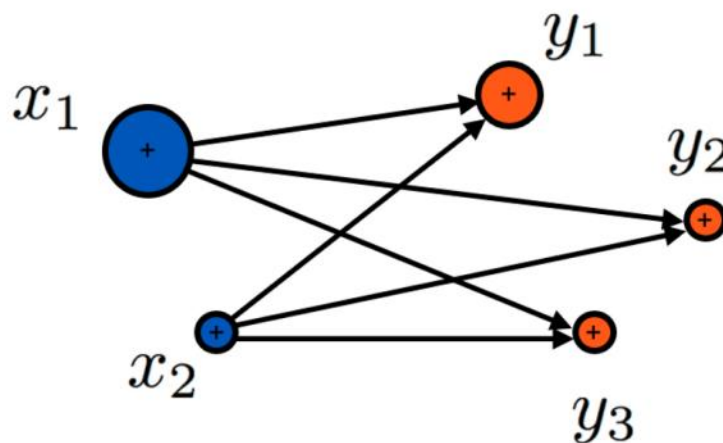
is known as Monge's problem

- however, it is a very strict formulation
  - ❖ it is not symmetric in the two distributions
  - ❖ it does not handle cases like the right picture



# Kantorovich's relaxation

- if we relate  $p$  and  $q$  using  $T$ , then intuitively a source point can only be assigned to a unique target point
- we can allow a “split” of the source point
- for two distributions, we assign a “transportation plan”



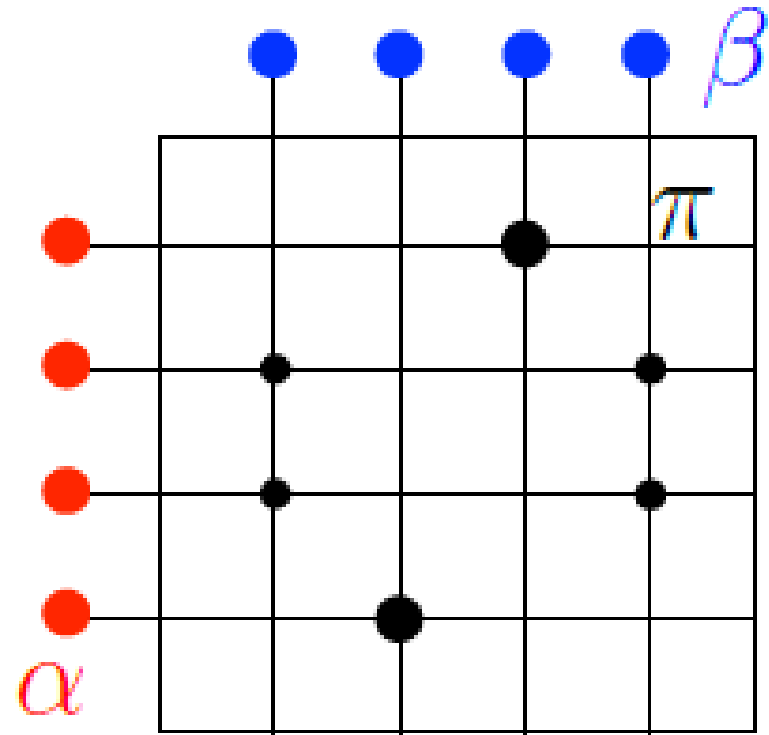
Resource allocation

# Kantorovich's relaxation

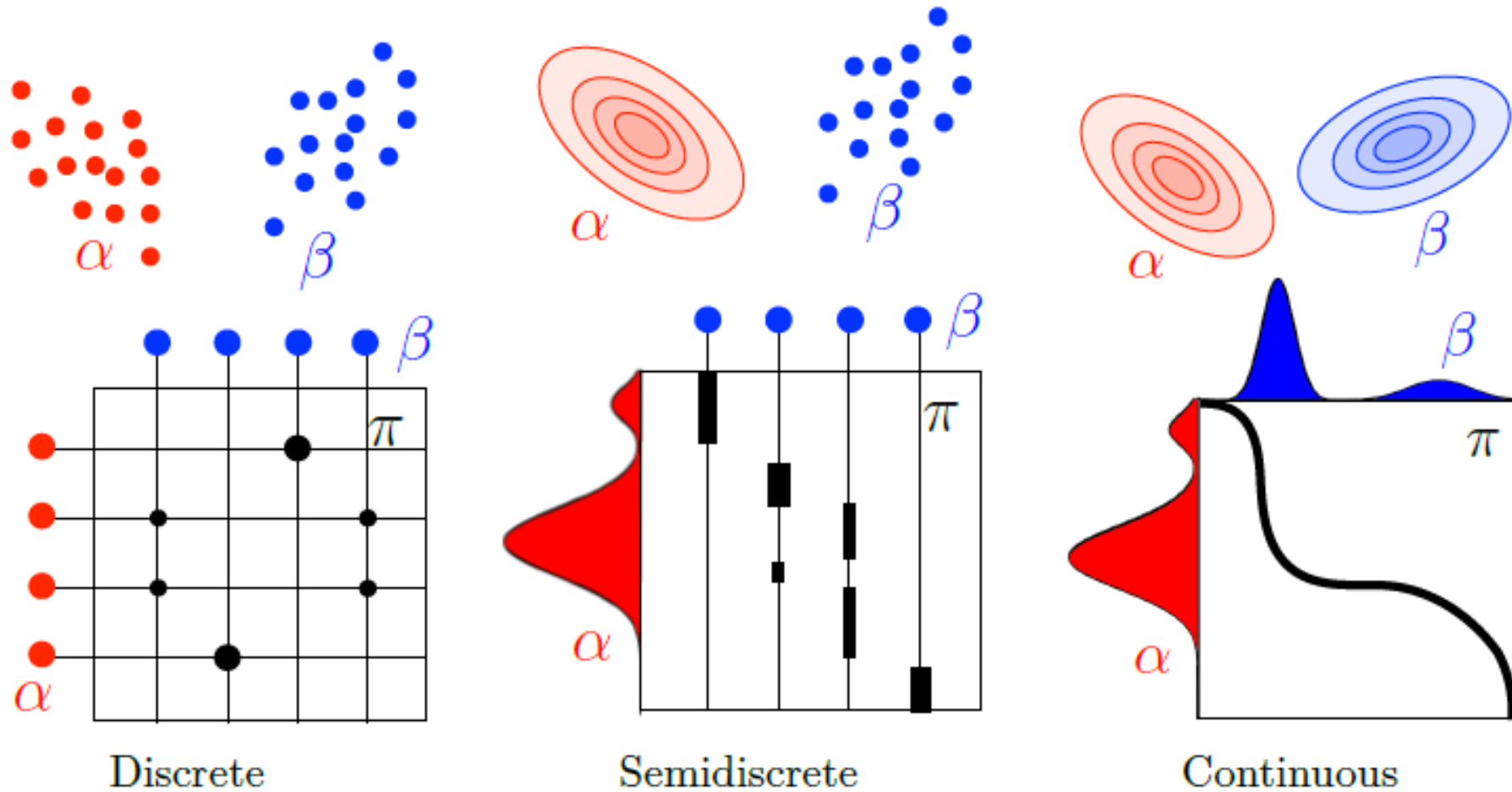
$$\begin{aligned} & \min_{\pi} \int c(x, y) \pi(x, y) dx dy \\ & \text{s.t.} \quad \int \pi(x, y) dy = p(x) \\ & \quad \int \pi(x, y) dx = q(y) \end{aligned} \quad \left. \vphantom{\int \pi(x, y) dy = p(x)} \right\} \begin{array}{l} \Pi(p, q) \\ \text{set of "couplings"} \end{array}$$

# Kantorovich's relaxation

$$\begin{aligned} \min_{\pi} \quad & \int c(x, y) \pi(x, y) dx dy \\ \text{s.t.} \quad & \int \pi(x, y) dy = p(x) \\ & \int \pi(x, y) dx = q(y) \end{aligned}$$



# Kantorovich's relaxation



## Wasserstein distance

$$W_n(p, q) = \left\{ \min_{\pi \in \Pi(p, q)} \int \|x - y\|^n \pi(x, y) dx dy \right\}^{1/n}$$

# Kantorovich-Rubinstein duality

- define the Lipschitz norm by

$$\|f\|_{\text{Lip}} = \sup_{x \neq y} \frac{|f(x) - f(y)|}{\|x - y\|}$$

- then

$$W_1(p, q) = \sup_{\|f\|_{\text{Lip}} \leq 1} \mathbb{E}_{x \sim p(x)} f(x) - \mathbb{E}_{x \sim q(x)} f(x)$$

- TBP later ...

# Wasserstein GAN

- Can we design a GAN from

$$W_1(p, q) = \sup_{\|f\|_{\text{Lip}} \leq 1} \mathbb{E}_{x \sim p(x)} f(x) - \mathbb{E}_{x \sim q(x)} f(x) \quad ?$$

# Wasserstein GAN

$$W_1(p, q) = \sup_{\|f\|_{\text{Lip}} \leq 1} \mathbb{E}_{x \sim p(x)} f(x) - \mathbb{E}_{x \sim q(x)} f(x)$$

- How do we impose the Lipschitz condition?

$$w \leftarrow \text{clip}(w, -c, c)$$

- ❑ advantage: easy to implement
- ❑ disadvantage: reduced model expressivity

# Wasserstein GAN

---

**Algorithm 1** WGAN, our proposed algorithm. All experiments in the paper used the default values  $\alpha = 0.00005$ ,  $c = 0.01$ ,  $m = 64$ ,  $n_{\text{critic}} = 5$ .

---

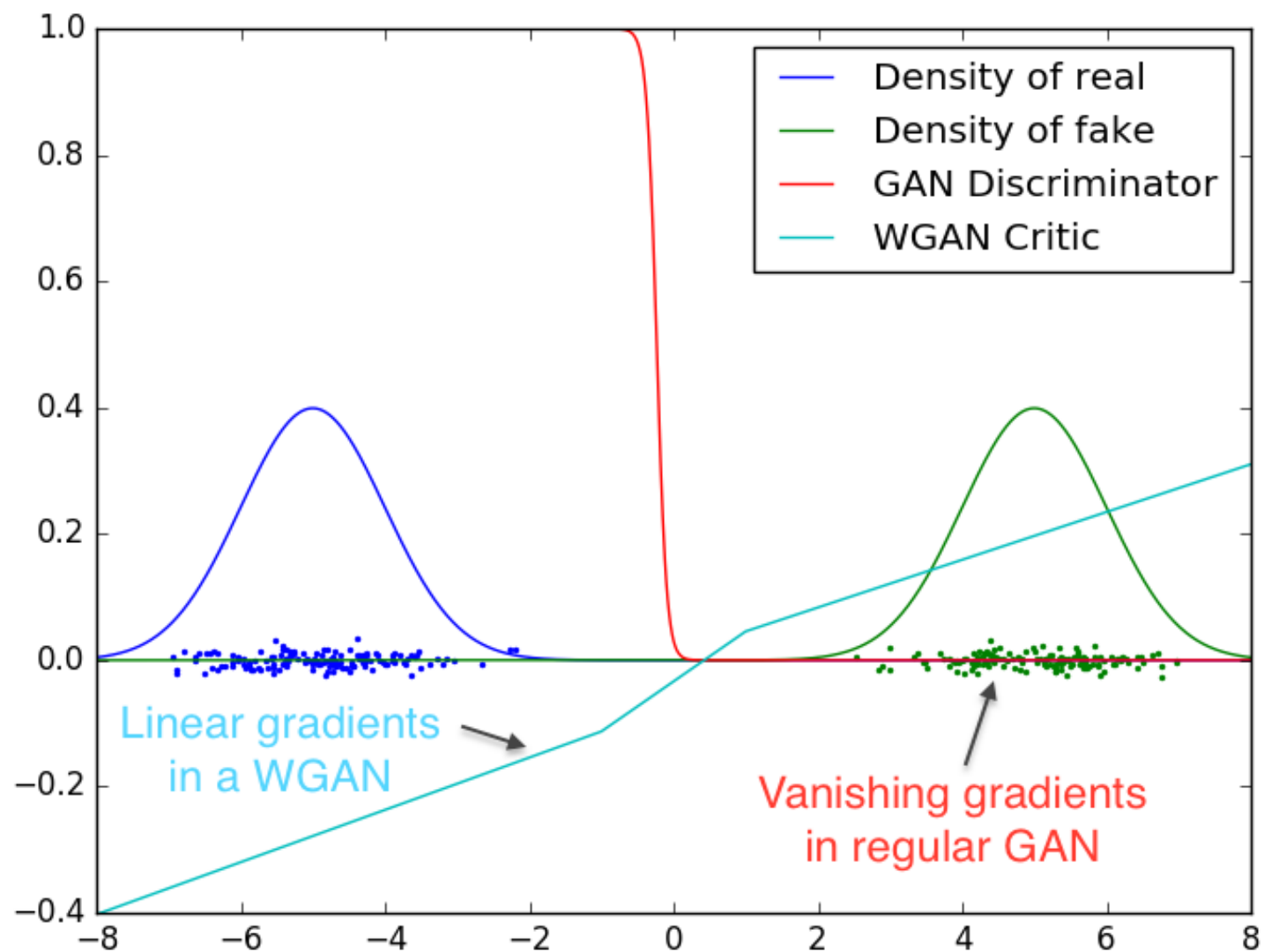
**Require:** :  $\alpha$ , the learning rate.  $c$ , the clipping parameter.  $m$ , the batch size.  $n_{\text{critic}}$ , the number of iterations of the critic per generator iteration.

**Require:** :  $w_0$ , initial critic parameters.  $\theta_0$ , initial generator's parameters.

```
1: while  $\theta$  has not converged do
2:   for  $t = 0, \dots, n_{\text{critic}}$  do
3:     Sample  $\{x^{(i)}\}_{i=1}^m \sim \mathbb{P}_r$  a batch from the real data.
4:     Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
5:      $g_w \leftarrow \nabla_w \left[ \frac{1}{m} \sum_{i=1}^m f_w(x^{(i)}) - \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)})) \right]$ 
6:      $w \leftarrow w + \alpha \cdot \text{RMSPProp}(w, g_w)$ 
7:      $w \leftarrow \text{clip}(w, -c, c)$ 
8:   end for
9:   Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
10:   $g_\theta \leftarrow -\nabla_\theta \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)}))$ 
11:   $\theta \leftarrow \theta - \alpha \cdot \text{RMSPProp}(\theta, g_\theta)$ 
12: end while
```

---

# Wasserstein GAN



# Wasserstein GAN

- in many cases WGAN is stabler and suffers less from mode collapse
- see some training tricks from <https://towardsdatascience.com/gan-ways-to-improve-gan-performance-acf37f9f59b>

## 8.3 proof of the Kantorovich-Rubinstein duality

# Kantorovich duality

- we first introduce a more general duality
- [[Kantorovich duality](#)] define  $I(\pi)$  and  $J(\phi, \psi)$  by

$$I(\pi) = \int c(x, y) \pi(x, y) dx dy$$
$$J(\phi, \psi) = \int \phi(x) p(x) dx + \int \psi(y) q(y) dy$$

then

$$\inf_{\pi \in \Pi(p, q)} I(\pi) = \sup_{(\phi, \psi) \in \Phi_c} J(\phi, \psi)$$

$$\Phi_c = \{(\phi, \psi) : \phi(x) + \psi(y) \leq c(x, y) \text{ for any } x, y\}$$

## proof of Kantorovich duality

- on one hand, it is easy to show 
$$\sup_{(\phi, \psi) \in \Phi_c} J(\phi, \psi) \leq \inf_{\pi \in \Pi(p, q)} I(\pi)$$

## proof of Kantorovich duality

- on the other hand, consider  $\inf_{\pi \in \Pi(p,q)} I(\pi)$

## proof of K-R duality

$$W_1(p, q) = \sup_{\|f\|_{\text{Lip}} \leq 1} \mathbb{E}_{x \sim p(x)} f(x) - \mathbb{E}_{x \sim q(x)} f(x)$$

- in view of the K duality,

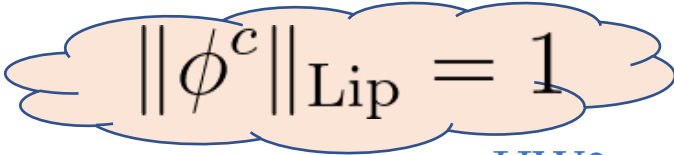
$$\sup_{(\phi, \psi) \in \Phi_c} J(\phi, \psi)$$

- define  $\phi^c(y) = \inf_x [c(x, y) - \phi(x)]$  .

$$\phi^{cc}(x) = \inf_y [c(x, y) - \phi^c(y)]$$

then

$$\sup_{(\phi, \psi) \in \Phi_c} J(\phi, \psi) = \sup_{\phi} J(\phi^{cc}, \phi^c)$$


$$\|\phi^c\|_{\text{Lip}} = 1$$

HW2

## proof of K-R duality

$$W_1(p, q) = \sup_{\|f\|_{\text{Lip}} \leq 1} \mathbb{E}_{x \sim p(x)} f(x) - \mathbb{E}_{x \sim q(x)} f(x)$$

- now that  $\|\phi^c\|_{\text{Lip}} = 1$ , for any  $x, y$ ,

$$\phi^c(y) - \phi^c(x) \leq \|x - y\|$$

- we have

$$-\phi^c(x) \leq \inf_y (\underbrace{\|x - y\| - \phi^c(y)}_{\phi^{cc}(x)}) \leq \|x - x\| - \phi^c(x) = -\phi^c(x)$$

## proof of K-R duality

$$W_1(p, q) = \sup_{\|f\|_{\text{Lip}} \leq 1} \mathbb{E}_{x \sim p(x)} f(x) - \mathbb{E}_{x \sim q(x)} f(x)$$

- therefore,

$$\begin{aligned} \sup_{(\phi, \psi) \in \Phi_c} J(\phi, \psi) &= \sup_{\phi} J(\phi^{cc}, \phi^c) = \sup_{\phi} J(-\phi^c, \phi^c) \\ &\leq \sup_{\|f\|_{\text{Lip}} \leq 1} J(f, -f) \leq \sup_{(\phi, \psi) \in \Phi_c} J(\phi, \psi) \end{aligned}$$

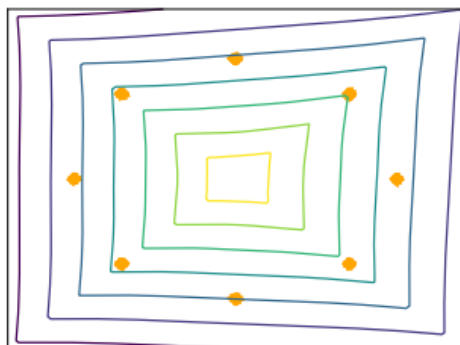
## 8.4 improved training of WGAN

## problem with weight clipping

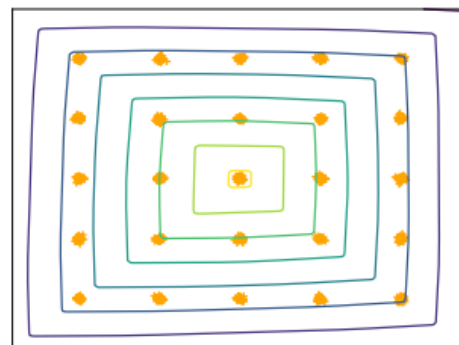
- to impose “Lip-1”, the original WGAN uses a weight-clipping trick to clip the weights of the discriminator to be within  $[-c, c]$
- this reduces expressivity of model

# problem with weight clipping

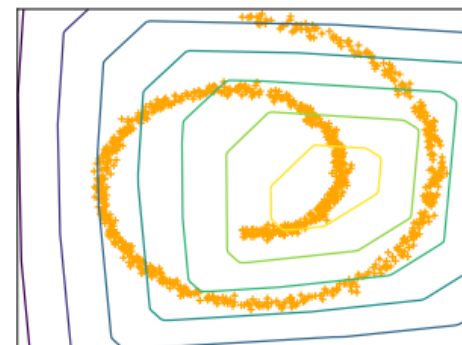
## 8 Gaussians



## 25 Gaussians

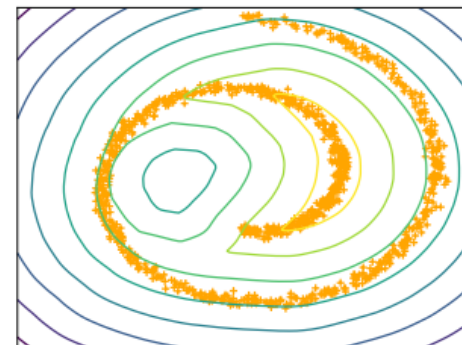
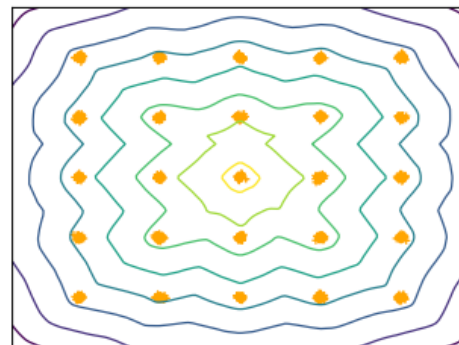
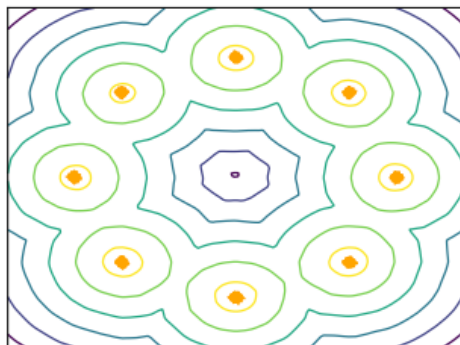


## Swiss Roll

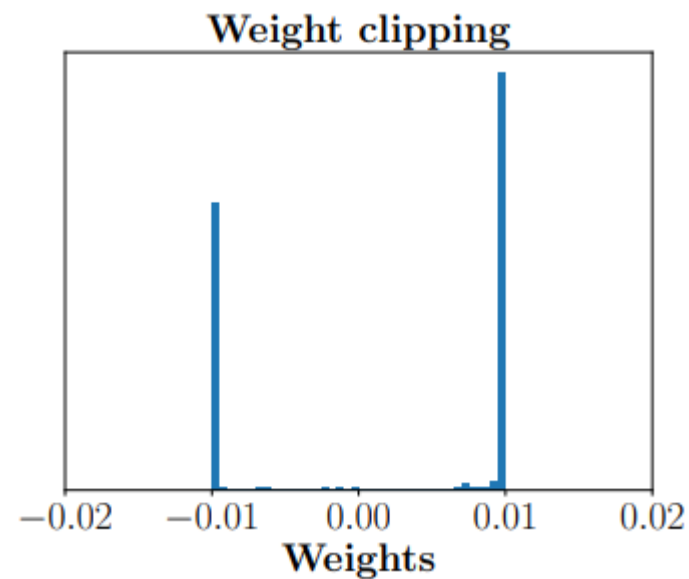
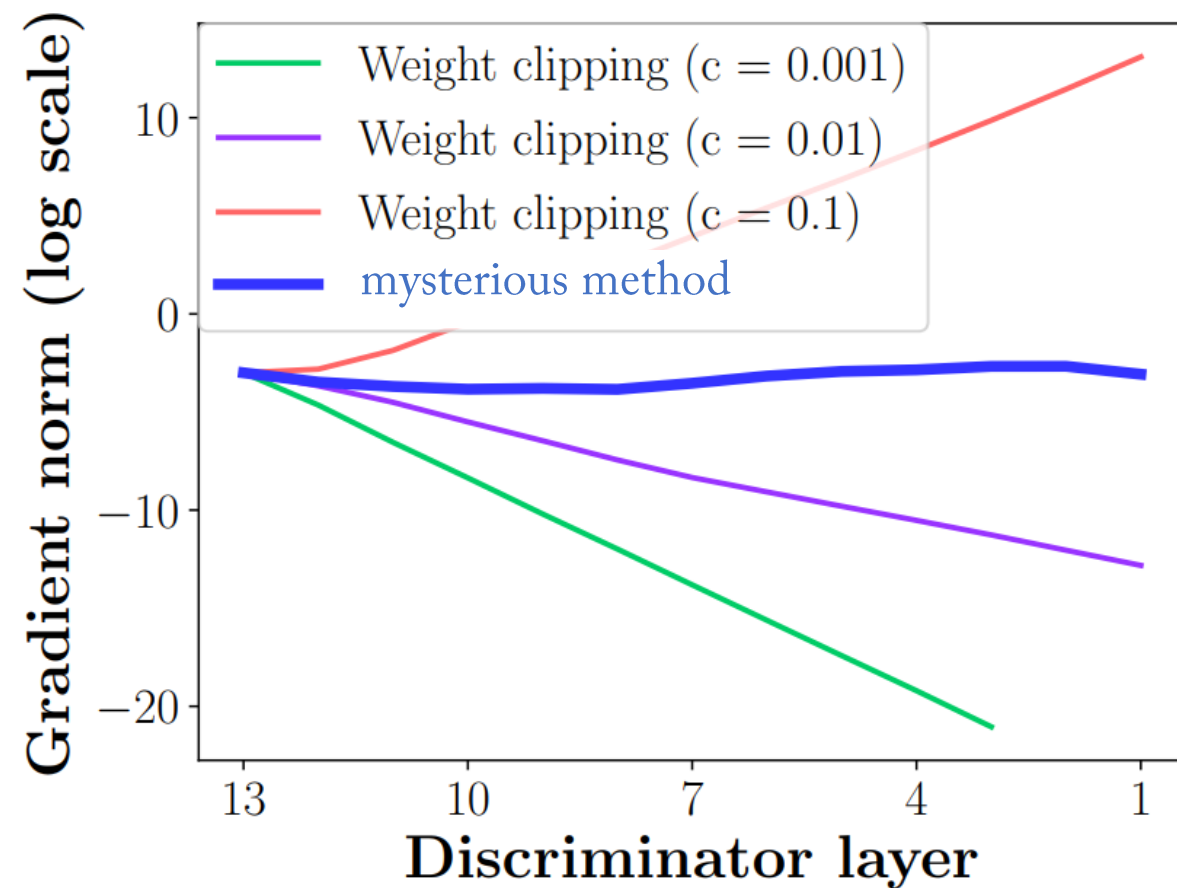


weight-clipping

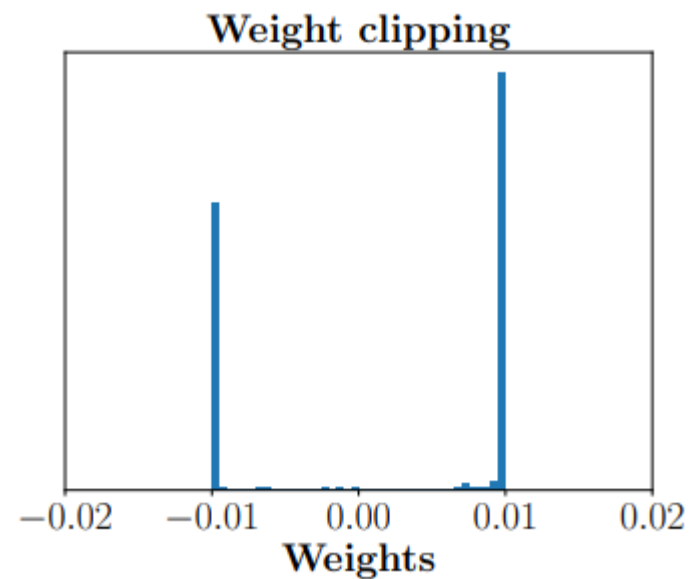
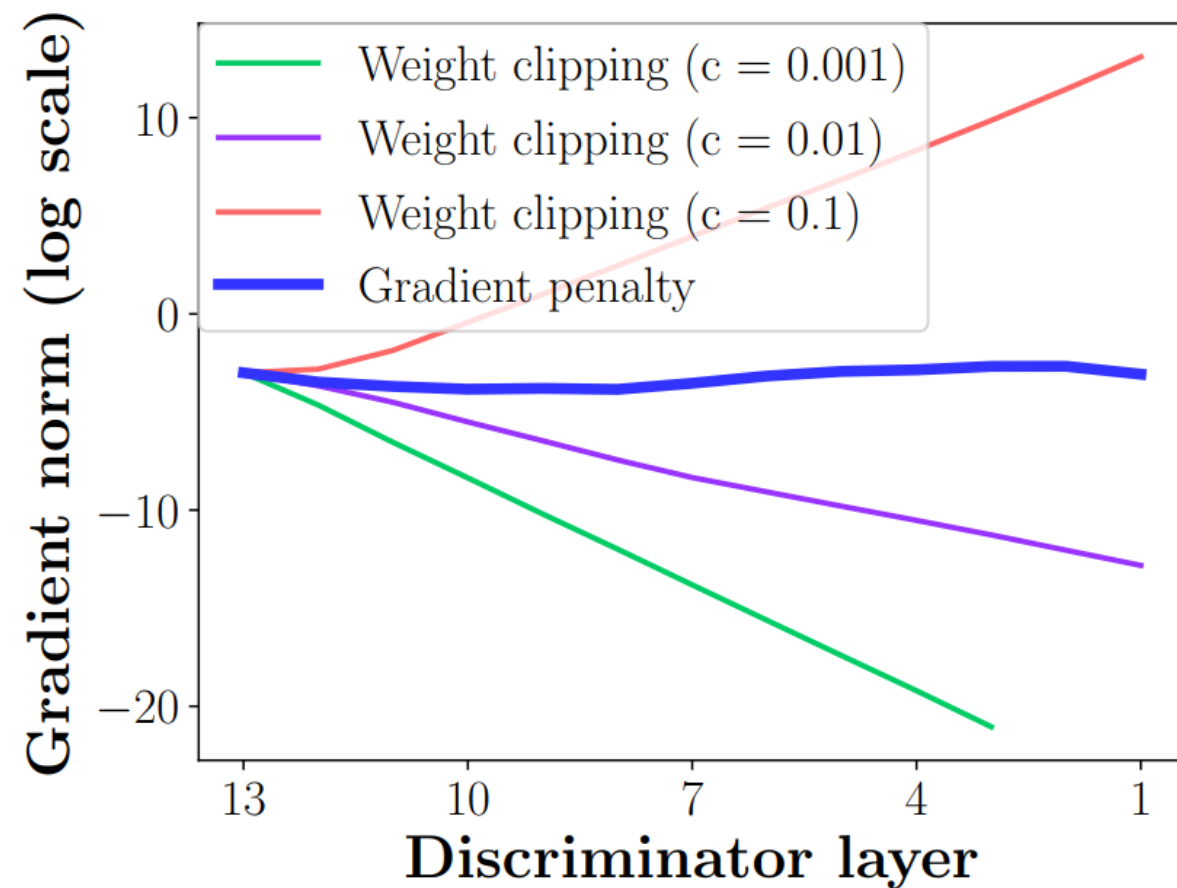
desired



# problem with weight clipping



# problem with weight clipping



# gradient penalty

$$L = \underbrace{\mathbb{E}_{\tilde{\mathbf{x}} \sim \mathbb{P}_g} [D(\tilde{\mathbf{x}})] - \mathbb{E}_{\mathbf{x} \sim \mathbb{P}_r} [D(\mathbf{x})]}_{\text{Original critic loss}} + \lambda \underbrace{\mathbb{E}_{\hat{\mathbf{x}} \sim \mathbb{P}_{\hat{\mathbf{x}}}} [(\|\nabla_{\hat{\mathbf{x}}} D(\hat{\mathbf{x}})\|_2 - 1)^2]}_{\text{Our gradient penalty}} .$$

sampled uniformly from the straight line connecting a sample from generation and a sample from training data

# WGAN-GP

---

**Algorithm 1** WGAN with gradient penalty. We use default values of  $\lambda = 10$ ,  $n_{\text{critic}} = 5$ ,  $\alpha = 0.0001$ ,  $\beta_1 = 0$ ,  $\beta_2 = 0.9$ .

---

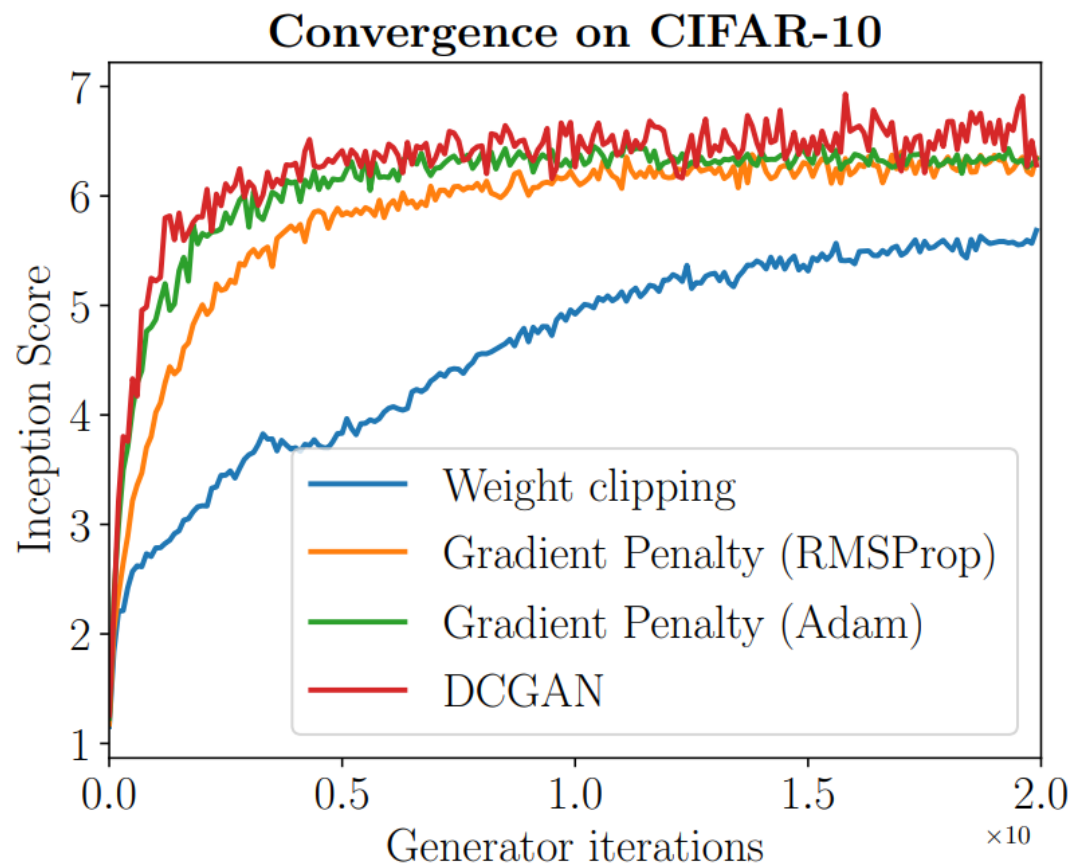
**Require:** The gradient penalty coefficient  $\lambda$ , the number of critic iterations per generator iteration  $n_{\text{critic}}$ , the batch size  $m$ , Adam hyperparameters  $\alpha, \beta_1, \beta_2$ .

**Require:** initial critic parameters  $w_0$ , initial generator parameters  $\theta_0$ .

```
1: while  $\theta$  has not converged do
2:   for  $t = 1, \dots, n_{\text{critic}}$  do
3:     for  $i = 1, \dots, m$  do
4:       Sample real data  $\mathbf{x} \sim \mathbb{P}_r$ , latent variable  $\mathbf{z} \sim p(\mathbf{z})$ , a random number  $\epsilon \sim U[0, 1]$ .
5:        $\tilde{\mathbf{x}} \leftarrow G_{\theta}(\mathbf{z})$ 
6:        $\hat{\mathbf{x}} \leftarrow \epsilon \mathbf{x} + (1 - \epsilon) \tilde{\mathbf{x}}$ 
7:        $L^{(i)} \leftarrow D_w(\tilde{\mathbf{x}}) - D_w(\mathbf{x}) + \lambda(\|\nabla_{\hat{\mathbf{x}}} D_w(\hat{\mathbf{x}})\|_2 - 1)^2$ 
8:     end for
9:      $w \leftarrow \text{Adam}(\nabla_w \frac{1}{m} \sum_{i=1}^m L^{(i)}, w, \alpha, \beta_1, \beta_2)$ 
10:   end for
11:   Sample a batch of latent variables  $\{\mathbf{z}^{(i)}\}_{i=1}^m \sim p(\mathbf{z})$ .
12:    $\theta \leftarrow \text{Adam}(\nabla_{\theta} \frac{1}{m} \sum_{i=1}^m -D_w(G_{\theta}(\mathbf{z}^{(i)})), \theta, \alpha, \beta_1, \beta_2)$ 
13: end while
```

---

# WGAN-GP



# Why sample from interpolation?

- fact: Let  $f^*$  be an optimal solution of

$$\max_{\|f\|_L \leq 1} \mathbb{E}_{y \sim \mathbb{P}_r}[f(y)] - \mathbb{E}_{x \sim \mathbb{P}_g}[f(x)]$$

if  $f^*$  is differentiable,  $\pi(x = y) = 0$ , and  $x_t = (1 - t)x + ty$  then it holds that

$$\mathbb{P}_{(x,y) \sim \pi} \left[ \nabla f^*(x_t) = \frac{y - x_t}{\|y - x_t\|} \right] = 1$$

proof of the fact

$$\mathbb{P}_{(x,y) \sim \pi} \left[ \nabla f^*(x_t) = \frac{y - x_t}{\|y - x_t\|} \right] = 1$$

- first, from  $\max_{\|f\|_L \leq 1} \mathbb{E}_{y \sim \mathbb{P}_r} [f(y)] - \mathbb{E}_{x \sim \mathbb{P}_g} [f(x)]$ , it holds that

$$\mathbb{P}_{(x,y) \sim \pi} [f^*(y) - f^*(x) = \|y - x\|] = 1$$

- let  $\psi(t) = f^*(x_t) - f^*(x)$

$$\begin{aligned} |\psi(t) - \psi(t')| &= \|f^*(x_t) - f^*(x_{t'})\| \\ &\leq \|x_t - x_{t'}\| \\ &= |t - t'| \underbrace{\|x - y\|}_{\text{Lip-norm of } \Psi} \end{aligned}$$

Lip-norm of  $\Psi$

proof of the fact

$$\mathbb{P}_{(x,y) \sim \pi} \left[ \nabla f^*(x_t) = \frac{y - x_t}{\|y - x_t\|} \right] = 1$$

- this implies

$$\begin{aligned} \psi(1) - \psi(0) &= \psi(1) - \psi(t) + \psi(t) - \psi(0) \\ \text{has to be "="} \left\{ \begin{aligned} &\leq (1-t)\|x-y\| + \psi(t) - \psi(0) \\ &\leq (1-t)\|x-y\| + t\|x-y\| \\ &= \|x-y\| \end{aligned} \right. \end{aligned}$$

- therefore,

$$\psi(t) = t\|x-y\|$$

proof of the fact

$$\mathbb{P}_{(x,y) \sim \pi} \left[ \nabla f^*(x_t) = \frac{y - x_t}{\|y - x_t\|} \right] = 1$$

• let

$$\begin{aligned} v &= \frac{y - x_t}{\|y - x_t\|} \\ &= \frac{y - ((1 - t)x + ty)}{\|y - ((1 - t)x + ty)\|} \\ &= \frac{(1 - t)(y - x)}{\|(1 - t)\| \|y - x\|} \\ &= \frac{y - x}{\|y - x\|} \end{aligned}$$

proof of the fact

$$\mathbb{P}_{(x,y) \sim \pi} \left[ \nabla f^*(x_t) = \frac{y - x_t}{\|y - x_t\|} \right] = 1$$

- the directional derivative

$$\begin{aligned} \frac{\partial}{\partial v} f^*(x_t) &= \lim_{h \rightarrow 0} \frac{f^*(x_t + hv) - f^*(x_t)}{h} \\ &= \lim_{h \rightarrow 0} \frac{f^* \left( x + t(y - x) + \frac{h}{\|y - x\|} (y - x) \right) - f^*(x_t)}{h} \\ &= \lim_{h \rightarrow 0} \frac{f^* \left( x_t + \frac{h}{\|y - x\|} (y - x) \right) - f^*(x_t)}{h} \\ &= \lim_{h \rightarrow 0} \frac{f^*(x) + \left( t + \frac{h}{\|y - x\|} \right) \|x - y\| - (f^*(x) + t\|x - y\|)}{h} \\ &= \lim_{h \rightarrow 0} \frac{h}{h} \\ &= 1 \end{aligned}$$

proof of the fact

$$\mathbb{P}_{(x,y) \sim \pi} \left[ \nabla f^*(x_t) = \frac{y - x_t}{\|y - x_t\|} \right] = 1$$

- therefore,

$$\begin{aligned} 1 &\leq \|\nabla f^*(x)\|^2 \\ &= \langle v, \nabla f^*(x_t) \rangle^2 + \|\nabla f^*(x_t) - \langle v, \nabla f^*(x_t) \rangle v\|^2 \\ &= \left| \frac{\partial}{\partial v} f^*(x_t) \right|^2 + \|\nabla f^*(x_t) - v \frac{\partial}{\partial v} f^*(x_t)\|^2 \\ &= 1 + \underbrace{\|\nabla f^*(x_t) - v\|^2}_{= 0, \text{ and we are done!}} \\ &\leq 1 \end{aligned}$$

# Thank you!

## Reference

---

- Ch 20.10.4, *Deep Learning*.
- Goodfellow, I., ... & Bengio, Y. (2014). Generative adversarial nets. *NIPS*.
- Nowozin, S., Cseke, B., & Tomioka, R. (2016). f-GAN: Training generative neural samplers using variational divergence minimization. *NIPS*.
- Arjovsky, M., & Bottou, L. (2017). Towards principled methods for training generative adversarial networks. *ICLR*.
- Arjovsky, M., Chintala, S., & Bottou, L. (2017). Wasserstein generative adversarial networks. *ICML*.
- Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., & Courville, A. (2017). Improved training of Wasserstein GANs. *NIPS*.
- Peyré, G., & Cuturi, M. (2019). Computational optimal transport: With applications to data science. *Foundations and Trends® in Machine Learning*, 11(5-6), 355-607.

